

LLM Evaluation & Observability Harness

Automated Quality Observability, Regression Testing & Caching Dashboard

Author: Eklavya (Final-Year CS Placement Portfolio)
Tech Stack: FastAPI (Python) & Next.js App Router (TypeScript)
Repository: github.com/eklavya434/llm-harness
Deploys: Hugging Face Spaces (Backend) & Vercel (Frontend)

1. The Core Problem: Why This Project Exists

Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) pipelines are increasingly used for chatbots, summarization, and search engines. However, LLMs are notoriously prone to **hallucinations**—generating factually incorrect answers that look convincing. Evaluating AI output quality is hard because traditional software testing (unit tests checking exact matches) does not work on variable, open-ended natural language responses.

Checking outputs manually is too slow, expensive, and subjective. This creates a critical need for automated, quantitative observability tools that grade LLM outputs, detect quality regressions during system updates, and log evaluation history.

2. The Solution: What This Project Does

This project implements a complete, self-contained LLM/RAG quality evaluation and regression testing platform. It acts as an automated **Quality Inspector** by grading chatbot outputs on a 1-5 scale across three rubrics, generating line trend visualizations, and enabling side-by-side regression testing when developers deploy new releases.

The evaluation is done using a dual method approach:

- 1. LLM-as-a-Judge:** The system sends inputs to Gemini 2.0 Flash (with Ollama as a local fallback). It instructs the LLM to grade the response according to strict definitions and return a structured JSON object containing a 1-5 score and a clear, one-line justification for that score.
- 2. Local Vector Embedding Similarity:** It measures semantic overlap by calculating the cosine similarity between the question and model response embeddings locally (on CPU) using the *all-MiniLM-L6-v2* model, incurring zero API latency or cost.

3. System Architecture & Tech Stack

The project is structured as a monorepo consisting of a decoupled FastAPI backend API and a Next.js frontend dashboard.

Component	Technology	Purpose
Backend Framework	FastAPI (Python)	Exposes REST APIs for runs, evaluations, comparisons, and health checks.
LLM Judge Client	google-genai SDK	Integrates Gemini 2.0 Flash with response_schema validations.
Database	SQLite (sqlite3)	Stores runs metadata, samples scores, and the judge cache table.
Embeddings Model	sentence-transformers	all-MiniLM-L6-v2 CPU-only embedding computations.
Frontend Portal	Next.js (App Router)	Sleek, modern user dashboard built with TypeScript.
UI Styling	Tailwind CSS	Premium obsidian-slate dark theme layout.
Data Visuals	Recharts	Interactive performance trend line charts.

4. Core Evaluation Rubrics

Faithfulness (1-5): Measures if the response is fully supported by the retrieved context. If the response introduces facts or claims not found in the context, it receives a 1 (hallucination). A score of 5 means it is 100% faithful.

Relevance (1-5): Evaluates if the response directly addresses the user's question. A low score is given if the chatbot gives off-topic information or avoids the core query.

Clarity (1-5): Checks if the response is grammatically correct, well-structured, coherent, and easy to comprehend.

5. Core Code Modules Explained

app/judge.py (The Evaluation Engine)

This file defines the prompt templates for each rubric, validates response structure using Pydantic, calculates cosine similarities with PyTorch, and enforces exponential backoff. It implements the primary Gemini evaluate call:

```
response = client.models.generate_content( model='gemini-2.0-flash', contents=prompt,
config=types.GenerateContentConfig( response_mime_type='application/json',
response_schema=RubricResponse, temperature=0.0 ) )
```

app/database.py (The Data Schema)

Creates three tables: **runs** (contains run summary scores and aggregates), **samples** (individual question details, outputs, and scores), and **judge_cache** (MD5 hashes mapping inputs to pre-computed scores). It also implements the regression comparison logic, joining matching queries between two versions and returning a delta report.

app/routes.py (The REST Interface)

Exposes backend routes: *POST /evaluate* (accepts a batch of samples, evaluates them, caches outputs, and saves the run), *GET /runs* (lists run history), *GET /runs/{run_id}* (retrieves run details), *GET /compare* (version-diff report), and *GET /quota-usage* (transparency metrics).

seed_db.py (Survival Seeder)

Wipes and rebuilds tables on startup if empty. It imports pre-computed evaluation cache results from a JSON file and recreates three mock runs (`v1.0.0`, `v1.1.0`, `v1.2.0`), so the dashboard has historical comparison data immediately upon server boot without utilizing live API keys.

6. Smart Production Engineering

Resilience to Ephemeral Storage (Hugging Face Spaces)

Hugging Face Spaces hosting uses read-only Docker containers except for the `/tmp` directory. Additionally, HF container environments are ephemeral—every time the Space goes to sleep and restarts, the database file ``/tmp/eval.db`` is completely wiped.

To solve this, we bundled pre-computed judge cache evaluations inside a committed JSON file (``data/judge_cache_seed.json``). On startup, ``seed_db.py`` runs, checks if the SQLite db exists, creates tables, and populates the database with the pre-evaluated metadata. This ensures that the demo dashboard is populated with historical data and functions perfectly on every boot.

Zero-Cost Demo Caching

To showcase a live LLM evaluation feature to recruiters without having your Gemini API quota exhausted, we use MD5 input caching. The cache key is calculated by hashing the concatenated string of: `hash(question + context + output)`. When a run is triggered on our frontend, the judge engine intercepts the call, finds the hash matches, and immediately returns the pre-computed scores from the SQLite cache table, resulting in **zero live API calls** and instant responses.

Next.js Wakeup Check Overlay

Free tier containers on Hugging Face Spaces sleep after 48 hours of inactivity. The first request to a sleeping container triggers a wake-up, taking about 30 seconds. To manage this user experience gracefully, our Next.js frontend implements a *WakeupCheck* wrapper. If the backend fails to respond, it renders a loading screen explaining the sleeping container mechanic and polls the API health endpoint until the server is alive, avoiding UI crashes.

7. Placement Interview Q&A; Guide

Here are 5 common questions technical interviewers will ask you about this project, and how you should answer them:

Q1: What is the benefit of using an LLM-as-a-Judge over traditional metrics like ROUGE or BLEU?

A: ROUGE and BLEU check for exact word overlap. If a model generates a perfect response but uses synonyms, ROUGE gives a low score. An LLM-as-a-judge understands semantics, logic, and contradictions. It can detect if the model introduced new claims not found in the context (hallucinations), which word-matching algorithms completely fail to do.

Q2: Why did you use Gemini 2.0 Flash as your primary judge, and how do you ensure structured outputs?

A: Gemini 2.0 Flash is extremely fast, cheap, and supports structured JSON outputs natively. By passing a Pydantic schema to the google-genai SDK's *response_schema* configuration, we force the Gemini API to return a strict, syntactically valid JSON object matching our score/reason layout, eliminating the need for regex parsers.

Q3: How does your database caching mechanism work?

A: We compute an MD5 hash of the concatenation of question + context + output. This forms a unique fingerprint for each evaluated sample. Before calling the Gemini API, we query our SQLite *judge_cache* table. If the fingerprint exists, we return the cached scores instantly. If not, we call Gemini, evaluate, and write the new result into the cache table.

Q4: How did you solve the ephemerality problem of Hugging Face Spaces?

A: Since HF Spaces only allows writes in `/tmp` and wipes files on container restart, we created `seed_db.py`. On startup, it initializes the database schema, imports pre-calculated evaluations from a committed JSON cache file, and registers our baseline runs. This ensures recruiters always see data on the dashboard, even after container restarts.

Q5: What is the purpose of the Regression Analysis feature in your dashboard?

A: Regression analysis compares two versions of a pipeline side-by-side. If we tweak the system (e.g. change chunking size, rewrite prompts), we run evaluation on a standard dataset, select v1 (older) and v2 (newer) in the dashboard, and immediately see if the average scores improved, degraded, or remained unchanged, allowing developers to identify quality regressions before release.